电子系统设计课程报告

标　　题　**迷宫小车及其上位机系统的设计**

组员姓名　**石殷睿　杨雨欣　于潇**

学　　号　**2228410216/2228410248/2029401073**

年级专业班级　**22电信 1班**

日　　期　2025年12月27日

苏州大学电子信息学院

目录

摘要 1
一. 设计系统方案 2
1.1 项目概述及设计目标 2
1.2 方案比较与关键器件选型 2
1. 算法方案选择 2
2. 红外方案选择 3
3. 陀螺仪方案选择 4
4. 主控方案选择 4
5. 电机方案选择 5
6. 供电方案选择 6
7. 总体设计框图 6
二. 理论分析与计算 7
2.1 陀螺仪姿态解算原理 7
2.2 电机控制理论分析 7
1. PWM 调速原理 7
2. 编码器速度反馈机制 7
3. 距离测量与堵转检测 7
2.3 PID 控制算法理论 8
1. 速度环 PID 控制 8
2. 角度环 PID 控制 8
3. 多级 PID 叠加与平衡修正 8
2.4 洪水算法路径规划理论 8
1. 算法基本原理 8
2. 算法复杂度与性能 8
三. 电路与软件设计 9
3.1 硬件设计 9
1. 核心板设计 9
8. 扩展板设计 9
9. 驱动板设计 10
3.2 软件设计 10
1. 小车控制代码 10
10. 上位机代码 13
四. 测试方案与测试结果 16
4.1 测试方案 16
4.2 测试结果 16
五. 结论 17
参考文献 18
附录 19
1. 硬件工程文件： 19
2. 小车源代码: 20
3. 上位机源代码: 21

迷宫小车设计

摘要

本设计实现了一个基于 STM32F103 微控制器的智能鼠（Micromouse）走迷宫系统。系统采用双轮差速驱动结构，通过红外传感器实时检测迷宫墙壁信息，利用洪水算法（Flood Fill Algorithm）进行路径规划，并采用多级 PID 控制算法实现精确的运动控制。

系统硬件部分包括 STM32F103 主控芯片、JY61P 陀螺仪模块、红外数字传感器、编码器、电机驱动电路等。软件部分采用模块化设计，实现了动作封装（直行、左转、右转、调头、停止）、洪水算法路径规划、死胡同检测与处理等核心功能。系统通过速度环 PID 和角度环 PID 的双级控制结构，结合速度环左右平衡修正，实现了稳定的直行和精确的转向控制。

系统配套开发了基于 PyQt5 的上位机监控软件，实现了实时轨迹可视化、传感器数据监控、迷宫墙体自动绘制、轨迹回放、路径优化等功能。上位机通过 UART3 串口与下位机通信，采用统一的数据协议，实现了完整的人机交互和调试功能。

测试结果表明，系统能够成功识别迷宫结构，规划最优路径，并准确执行前进、转向等动作。智能鼠能够从起点（7,0）出发，通过洪水算法计算到中心区域的最短路径，并在遇到死胡同时自动返回。上位机软件能够实时显示智能鼠的运动轨迹和传感器状态，为系统调试和性能分析提供了有力支持。系统运行稳定，控制精度满足设计要求，成功实现了预期的自动走迷宫功能。

关键词：迷宫小车；STM32F103RET6 单片机；红外测距；编码器；陀螺仪；上位机

一. 设计系统方案

1.1 项目概述及设计目标

迷宫小车项目，又称电脑鼠（MicroMouse）是一项以 8×8 方格迷宫为背景、终点设在坐标 (3,3) 与 (4,4) 中心点的智能微型机器人挑战：嵌入式微控制器驱动传感器与机电结构，让小车在未知墙壁中自主搜索、记忆最优路径，并以最短时间冲刺到指定终点；该项目为 IEEE 经典赛事，将机械、电子、控制与 AI 知识熔于一炉，为自动化及电子信息等专业学生提供贯通理论、实践与创新的综合练兵场。

本系统旨在设计并实现一个能够自主探索和穿越 8×8 迷宫的智能鼠系统。系统需要具备以下核心功能：

(1) **环境感知能力**：通过红外传感器实时检测前方、左侧、右侧的墙壁信息；
(2) **路径规划能力**：采用洪水算法计算从当前位置到迷宫中心的最短路径；
(3) **精确运动控制**：实现直行、90°左转、90°右转、180°调头等基本动作；
(4) **死胡同处理**：能够识别死胡同并自动返回；
(5) **姿态稳定控制**：通过陀螺仪反馈实现精确的角度控制和直行稳定。
(6) **状态实时监测**：通过上位机实现对小车传感器，小车运行姿态以及小车在迷宫中实时位置的监测，支持绘制出迷宫的墙壁及路线。

1.2 方案比较与关键器件选型

1. 算法方案选择

方案一：左手法则

这是最简单直观的迷宫探索方法，机器人始终沿着左侧墙壁前进，遇到死胡同时调头返回[1]。该方案的最大优势是实现简单、代码量小、内存占用极低(O(1)空间复杂度)，无需存储任何地图信息。但其致命缺陷在于无法保证路径最优，可能产生大量重复遍历，时间复杂度高达 $O(n^2)$，且在复杂迷宫结构中表现不佳。该方案仅适用于简单迷宫的教学演示或对时间效率要求不高的场景。



图 1 左手法则示意图

方案二：洪水算法

采用从目标点向外扩散的迭代松弛策略，为每个格子计算到中心的最短距离值，机器人沿距离递减方向前进[2]。该方案在保证路径最优性的同时，算法效率较高，时间复杂度为 $O(n^2 \times k)$，其中迭代次数 k 通常小于 10。虽然需要 $O(n^2)$的空间存储完整地图信息(对 8×8 迷宫为 64 个格子数据)，但在 STM32F103 平台上计算时间仅需 10ms 左右,完全满足实时性要求。该算法能够自动处理死胡同,支持墙信息动态更新后的路径重规划,非常适合标准 IEEE Micromouse 竞赛场景。

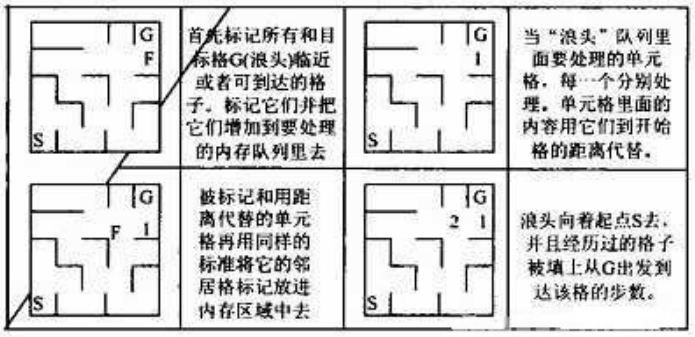


图 2 洪水算法

方案三：A*算法

这是一种基于启发式函数(曼哈顿距离)的智能搜索算法,能够优先探索最有希望的路径方向[3]。该方案在理论上路径最优且算法成熟,特别适合处理动态障碍物场景。但其缺点同样明显:需要维护开放列表和关闭列表,时间和空间复杂度均为 O(b^d),计算量大、内存占用高、实现复杂度高。对于资源受限的 STM32F103 平台和 8×8 标准迷宫而言,A*算法显得过于"重量级",更适合大型迷宫或高性能处理器平台。

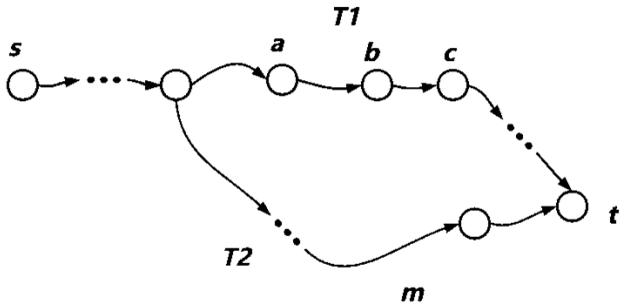


图 3 A*算法

方案选择理由 -综合考虑算法效率、实现复杂度和系统性能要求，选择方案二（**洪水算法**）。该算法在保证路径最优的同时，计算量适中，适合 STM32F103 平台。对于 8×8 标准迷宫，洪水算法的迭代次数通常不超过 10 次，计算时间在 10ms 以内，完全满足实时性要求。同时，洪水算法实现相对简单，代码可维护性好，适合教学和竞赛使用。

2. 红外方案选择

方案一：数字量红外传感器

采用内置比较器直接输出数字信号的传感器，通过 GPIO 读取 0/1 状态判断墙体存在。该方案最大优势是响应速度快(<1ms)、电路简单、抗干扰能力强、成本低(约 3 元/个)且无需软件阈值标定。但其致命缺陷是无法获取距离信息,只能简单判断有墙/无墙,检测距离固定需硬件调节电位器,无法实现墙距居中控制。该方案仅适用于对距离精度要求不高的简单迷宫检测场景。

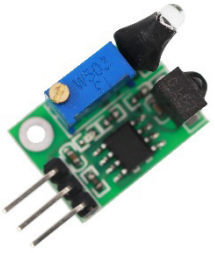


图 4 BZ001 数字量红外传感器

方案二：模拟量红外传感器

输出模拟电压信号,通过 ADC 采样获取精确距离值。该方案能够获取精确距离信息,支持墙距居中控制提高直行精度,检测距离可软件调节,支持多级阈值判断。但其缺点是需要 ADC 采样导致响应时间较长(>10ms),电路相对复杂需要占用 ADC 通道,受电源波动影响需要稳定电源,需要软件阈值标定,成本稍高(约 5-8 元/个)。该方案适合需要精确距离控制的高级迷宫算法场景。

方案选择理由 -综合考虑系统复杂度、开发效率和实际需求,**推荐选择方案一(数字量红外传感器)**。对于 8×8 标准迷宫机器人，墙体检测的核心需求是快速准确地判断"有墙"或"无墙"，而非精确测距。数字量传感器的<1ms 响应速度远优于模拟量方案(>10ms)，能够更及时地捕捉墙体信息，提高系统实时性。配合 JY61P 陀螺仪提供的高精度姿态角(±0.5°)进行方向控制，完全可以实现直线行驶和精确转向，无需依赖墙距居中算法。

3. 陀螺仪方案选择

方案一：MPU6050 六轴传感器

集成三轴加速度计和三轴陀螺仪,通过 I2C 通信。该方案成本低(约 5-10 元)、集成度高、体积小、数据更新频率高(最高 1kHz)。但其最大问题是需要自行开发软件融合算法(如互补滤波、卡尔曼滤波)来计算姿态角,开发复杂度高,精度受算法实现影响,需要大量调试参数,且存在漂移问题需要定期校准。该方案适合对成本敏感且具备算法开发能力的项目。

方案二：JY61P 姿态传感器

集成 MPU6050 芯片和姿态解算算法,直接通过 UART 输出欧拉角。该方案内置姿态解算算法直接输出 Roll/Pitch/Yaw 角度,无需自行开发融合算法,开发极为简单。技术指标优异:静态精度±0.1°、动态精度±0.5°、分辨率 0.0055°、更新频率 100Hz,数据稳定漂移小。UART 通信(115200 波特率)简单可靠,数据格式清晰。唯一缺点是成本稍高(约 15 元)且需要占用一个 UART 资源,但相比开发复杂度的大幅降低,这点成本完全值得。



图 5 JY61P 姿态传感器

方案选择理由 -综合考虑开发效率、精度要求和成本,**选择方案二(JY61P)**。该传感器最大优势是"开箱即用",内置的姿态解算算法让开发者无需深入研究复杂的传感器融合理论,直接获取高精度角度信息。对于需要快速开发和稳定运行的迷宫机器人,JY61P 的±0.5°动态精度和 100Hz 更新率完全满足实时转向控制需求。虽然比 MPU6050 贵 10 元,但节省的开发时间和调试成本远超这个差价,是性价比最优的选择。

4. 主控方案选择

方案一：STM32F103RET6

ARM Cortex-M3 内核,72MHz 主频,512KB Flash,64KB RAM。该方案性能适中完全满足实时控制需求,外设极为丰富(3 个 UART、2 个 ADC、多个定时器、编码器接口),开发资源丰富 HAL 库支持完善,成本低(约 25 元),功耗低适合电池供电,支持软件浮点运算。唯一缺点是无硬件浮点单元导致浮点运算速度较慢,Flash 和 RAM 容量有限,但对于 8×8 迷宫的洪水算法计算和

PID 控制,这些资源完全充足。

方案二：STM32F407 高性能系列

ARM Cortex-M4 内核,168MHz 主频,1MB Flash,192KB RAM。该方案性能强劲,硬件浮点单元使浮点运算极快,内存容量大。但成本高(约 50-80 元),功耗较高,对于迷宫机器人来说性能严重过剩。该方案更适合需要大量浮点计算或复杂算法的高性能应用,如图像处理或复杂运动控制。

方案三：Arduino 系列

基于 AVR 或 ARM 的简易开发板。该方案开发极为简单,库函数丰富,社区支持好,适合快速原型。但性能有限,实时性差,外设配置不灵活,不适合需要精确时序控制的应用。该方案仅适合原型开发和教学演示,无法满足竞赛级精度要求。

方案选择理由 - 综合考虑性能需求、开发成熟度和成本,**推荐选择方案一(STM32F103RET6)**。该芯片在国内嵌入式教学和竞赛中应用极广,开发资料完善,HAL 库成熟稳定。72MHz 主频对于执行洪水算法(<10ms)、多路 PID 控制(1kHz)、传感器数据采集等任务绰绰有余。丰富的外设接口(3 个 UART 用于陀螺仪/调试/扩展,2 个 ADC 用于传感器/电压监测,定时器支持编码器和 PWM)完美匹配系统需求。

5. 电机方案选择

方案一：步进电机

通过精确控制步进角度实现开环位置控制。该方案控制精度高无累积误差,低速扭矩大,位置控制精确。但速度慢不适合快速移动,需要步进驱动器成本高,功耗大不适合电池供电,控制复杂需要细分驱动。该方案更适合精密定位、低速高精度应用,如 3D 打印机、CNC 雕刻机,而非需要快速移动的迷宫机器人。

方案二：直流减速电机+编码器

直流电机配合减速箱和编码器反馈实现闭环控制。该方案速度可调范围大适合快速移动(100RPM 减速后输出),成本适中(约 18 元/个),PWM 调速控制简单,编码器反馈(500 线/圈,减速比 30:1,每圈 15000 计数)实现精确闭环控制,功耗适中适合电池供电,减速比可选平衡扭矩和速度[4]。缺点是需要编码器增加成本,需要实现 PID 控制算法,选用 GM12 电机自带编码器。但



图 6 GM12 直流减速电机

方案三：无刷电机(BLDC)

高效率长寿命的电机方案。该方案效率高功耗低,寿命长无电刷磨损,速度范围大。但成本高(约 50-100 元/个),需要专用 ESC 驱动器控制复杂,对于迷宫机器人来说性能过剩。该方案更适合高性能机器人、无人机等需要长时间高效运行的应用。

方案选择理由 - 综合考虑速度性能、控制精度和成本,**推荐选择方案二(直流减速电机+编码器)**。该方案在成本、性能和控制复杂度之间取得了最佳平衡。直流电机 PWM 调速实现 0-100RPM 连续可调,满足迷宫中直行和转弯的不同速度需求;500 线高分辨率编码器配合 30:1 减速比提供每圈 15000 计数的精确反馈,支持毫米级位移控制;通过速度环和位置环双环 PID 控制实现精确的路径跟踪。18 元/个的价格和适中功耗使其成为移动机器人的理想选择

6. 供电方案选择

方案一：锂电池+稳压

使用 18650 锂电池或聚合物锂电池(标称 7.4V 双节串联或 3.7V 单节),通过稳压电路提供 5V 输出。该方案能量密度高(859mAh 容量)续航时间长,重量轻适合移动机器人,可充电使用方便,成本适中。系统通过稳压输出 5V 给 STM32 和传感器供电 ,并通过 ADC 监测电池电压实现低电压保护。缺点是需要电池保护电路,需要电压监测,放电曲线不够平坦,但这些都可以通过成熟的电源管理方案解决。



图 7 2s 锂电池方案（改 XT-30 接口）

方案二：干电池供电

使用 AA 或 AAA 干电池串联。该方案成本低,更换方便,无需充电器。但能量密度低导致续航时间短,重量重影响机器人性能,电压下降明显影响控制稳定性,频繁更换不环保且不经济。该方案仅适合短期演示或教学项目,不适合需要长时间运行的竞赛应用。

方案三：外部电源供电

通过 USB 或外部适配器供电。该方案电压稳定,无需考虑续航问题,适合长时间调试。但需要连接线严重限制移动范围,完全不适合迷宫机器人的实际应用场景。该方案仅适用于开发调试阶段的固定位置测试。

方案选择理由 - 综合考虑能量密度、重量和使用便利性,**推荐选择方案一(锂电池+稳压)**。现代 18650 或聚合物锂电池技术成熟,850mAh 容量可支持机器人连续运行约 30 分钟,完全满足竞赛和演示需求。7.4V 标称电压通过 DC-DC 稳压提供稳定的 5V 逻辑电源,保证系统在整个放电过程中稳定运行。可充电特性大幅降低长期使用成本,轻量化设计不影响机器人运动性能。

7. 总体设计框图

总体设计框图如下图 8 所示。

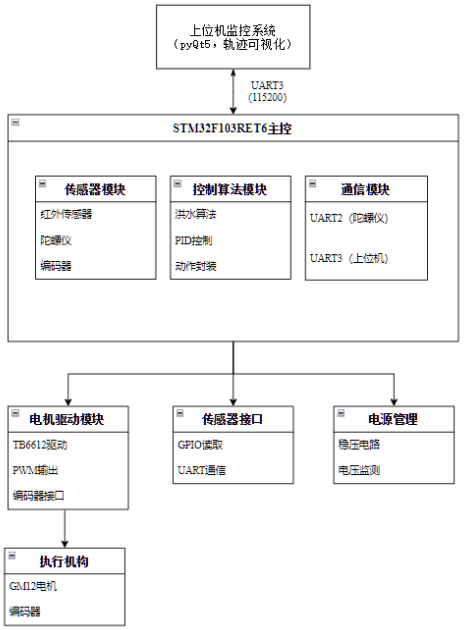


图 8 总体设计框图

二. 理论分析与计算

2.1 陀螺仪姿态解算原理

系统采用 JY61P 姿态传感器模块，该模块内置 MPU6050 六轴传感器和姿态融合算法，通过 UART 串口以 115200 波特率输出欧拉角数据。数据帧格式包含帧头、数据类型标识、六字节姿态数据（Roll、Pitch、Yaw 各两字节）和校验和。

$$\begin{cases} Roll = (int16_t)((Data[2] << 8) | Data[1]) \times 180.0/32768.0 \\ Pitch = (int16_t)((Data[4] << 8) | Data[3]) \times 180.0/32768.0 \\ Yaw = (int16_t)((Data[6] << 8) | Data[5]) \times 180.0/32768.0 \end{cases} \quad (1)$$

姿态数据采用 16 位有符号整数表示，范围为-32768 至 32767，通过公式（1）对应至-180° 至 180° 的角度范围。系统通过大端序解析数据，将原始数据乘以 180 除以 32768 转换为角度值。角度分辨率约为 0.0055 度，静态精度为±0.1 度，动态精度为±0.5 度，更新频率为 100 赫兹。系统在启动时将当前偏航角作为零位基准，在直行动作中首次进入时锁定当前偏航角作为目标角度，通过公式（2）角度归一化处理将角度限制在-180° 至 180° 范围内，确保角度误差计算的正确性。

$$NormalizeAngle(\theta) = \begin{cases} \theta - 360°, 当\theta > 180° \\ \theta + 360°, 当\theta < -180°\theta, 其他情况 \end{cases} \quad (2)$$

2.2 电机控制理论分析

1. PWM 调速原理

系统采用脉宽调制（PWM）技术控制直流电机转速，PWM 频率设置为 1 千赫兹，占空比范围 0%至 100%对应电机转速从零到最大。电机目标速度以编码器计数增量表示，范围为 20 至 120 计数每 50 毫秒，迷宫直行基础速度设置为 60 计数每 50 毫秒。系统通过定时器产生 PWM 信号，右轮使用 TIM1 通道 1，左轮使用 TIM4 通道 1，PWM 占空比由 PID 控制器输出决定。H 桥驱动电路根据方向控制信号和 PWM 信号控制电机正反转和转速，当方向控制信号为正向时电机前进，为反向时电机后退。系统通过编码器反馈实现闭环控制，编码器每 50 毫秒采样一次，计算速度增量，经过式（3）所示的一阶低通滤波抑制抖动后作为速度反馈信号。

$$\begin{cases} EncoderRight_filtilt(n) = EncoderRight_filt(n-1) + \alpha \times [EncoderRight - EncoderRight_filt(n-1)] \\ EncoderLeft_filt(n) = EncoderLeft_filt(n-1) + \alpha \times [EncoderLeft - EncoderLeft_filt(n-1)] \\ 其中：\alpha = 0.45 \end{cases} \quad (3)$$

2. 编码器速度反馈机制

系统采用增量式编码器，左右轮各配置一个 500 线编码器，通过定时器编码器模式自动计数。编码器每转产生 500 个脉冲，经过减速箱后每转输出 15000 个计数（假设减速比 30:1）。如式（4），系统每 50 毫秒读取编码器计数值，通过比较当前值与上次值的差值计算速度增量，并根据编码器计数方向判断前进或后退。为抑制编码器噪声，系统对速度进行一阶低通滤波，滤波系数为 0.45，滤波后的速度值作为 PID 控制器的反馈信号。系统还实现编码器死区处理，当速度增量绝对值小于 2 时视为零，避免微小抖动影响控制精度。

$$\Delta N = N(t) - N(t - Ts), Ts = 50ms \quad (4)$$

3. 距离测量与堵转检测

系统通过编码器累积值判断是否完成一个单元格的移动。**在实际迷宫中进行测量**，每个单元格对应 256 个编码器计数，如式（5），系统每 50 毫秒计算左右轮速度增量的平均值并累加，当累积值达到 256 时判定为完成一个单元格的移动，更新机器人坐标并停止运动。为检测堵转情况，系统实现超时检测机制：当前进时间超过 1.5 秒且编码器累积值小于单元格距离的八分之一时，判定为堵转，标记前方为墙并停止运动。

$$N_acc(k) = N_acc(k-1) + (|v_right| + |v_left|)/2 \quad (5)$$

2.3 PID 控制算法理论

1. 速度环 PID 控制

速度环 PID 控制器每 10 毫秒执行一次，以编码器反馈速度与目标速度的误差作为输入，输出 PWM 占空比调整量。比例系数 Kp 设置为 2.0，用于快速响应速度误差；积分系数 Ki 设置为 0.25，用于消除稳态误差；微分系数 Kd 设置为 1.8，用于抑制超调和振荡。为防止积分饱和，系统仅在误差绝对值小于 15 时累积积分项，大误差时积分项以 0.95 的系数衰减。积分项限幅为±300，输出限幅为±800，确保控制信号在合理范围内。系统还实现输出死区处理，当输出绝对值小于 5 时置零，防止电机在目标速度附近抖动。PID 输出公式如下：

$$\begin{cases} e(t) = MotorTarget - EncoderFilt \\ u(t) = Kp \cdot e(t) + Ki \cdot \int e(t)dt + Kd \cdot de(t)/dt PWM_D uty = clamp(u(t), -800, 800) \end{cases} \tag{6}$$

2. 角度环 PID 控制

角度环 PID 控制器叠加在速度环之上，用于保持直行方向或控制转向精度。直行时使用角度环 PID 保持方向稳定，比例系数 Kp 为 1.1，积分系数 Ki 为 0.004，微分系数 Kd 为 0.9，积分限幅±150，输出限幅±150。系统仅在角度误差小于 5 度时累积积分，输出经过平滑处理，变化量限幅为 4，抑制左右摆动。转向时使用独立的转向 PID 控制器，参数更柔和：Kp 为 0.9，Ki 为 0.0025，Kd 为 0.45，积分限幅±100，输出限幅±100。转向时还实现收敛减速机制，当角度误差小于 18 度时，输出按误差比例线性减小，误差越小输出越小，有效降低转向过冲。

3. 多级 PID 叠加与平衡修正

系统采用速度环和角度环两级 PID 叠加控制，速度环输出作为基础 PWM 值，角度环输出作为修正量叠加在速度环之上。直行时系统还实现速度环左右平衡修正，根据左右轮速度差计算修正量，修正系数为 0.35，最大修正量为±8，用于补偿左右轮机械差异导致的偏航。系统在每次动作切换时重置所有 PID 状态，包括积分项、上次误差和输出值，防止上一个动作的积分干扰新动作，确保控制系统的快速响应和稳定性。

2.4 洪水算法路径规划理论

1. 算法基本原理

洪水算法（Flood Fill Algorithm）是一种基于迭代松弛的最短路径搜索算法，从目标点（迷宫中心）开始向外扩散，为每个格子计算到中心的最短距离。算法初始化时将所有格子的距离值设为最大值 255，中心区域（3,3）至（4,4）的距离设为 0。

$$\begin{cases} \Delta x = 0, 1, 0, -1 (北、东、南、西) \\ \Delta y = 1, 0, -1, 0 \end{cases} \tag{7}$$

定义方向向量（7），然后通过迭代松弛过程（8），遍历所有格子，检查四个方向（北、东、南、西），如果某个方向无墙且邻居格子的距离加 1 小于当前格子的距离，则更新当前格子的距离值[5]。迭代过程持续进行直到达到收敛，直到所有格子的距离值不再变化，此时每个格子的距离值即为到中心的最短距离。

$$d'(x,y) = min(d(x,y), min[d(x+\Delta x_i, y+\Delta y_i)+1],\ i \in 可达方向 \tag{8}$$

$$\forall (x,y): d'(x,y) = d(x,y)\ (收敛) \tag{9}$$

2. 算法复杂度与性能

对于 8×8 的标准迷宫，洪水算法的时间复杂度为 $O(n^2 \times k)$，其中 n 为迷宫大小（8），k 为迭代次数，通常小于 10 次。空间复杂度为 $O(n^2)$，需要存储 64 个格子的距离值和墙信息。在 STM32F103 微控制器 72MHz 主频下，算法计算时间小于 10 毫秒，完全满足实时性要求。算法能够保证计算出的路径是最短路径，同时支持墙信息动态更新后的路径重规划，非常适合标准 IEEE Micromouse 竞赛场景。

三. 电路与软件设计

3.1 硬件设计

1. 核心板设计

如图 9，核心板基于 STM32 微控制器,是整个系统的控制中枢和决策大脑。它提供丰富的外设资源包括多路 PWM 输出用于电机速度控制、编码器接口实现闭环反馈、ADC 通道监测电池电压、多路 GPIO 处理传感器和按键信号,以及 UART/SPI/I2C 等通信接口[6]。核心板通过 24 针双排扩展接口与扩展主板连接接收传感器数据,通过 12 针排针与驱动板连接输出电机控制信号和接收编码器反馈,构成完整的"感知-决策-执行"控制回路。它负责运行控制算法(如 PID 速度控制、循迹算法、姿态解算等)、处理显示和通信任务、协调各模块工作,是整个智能小车系统的核心控制单元。

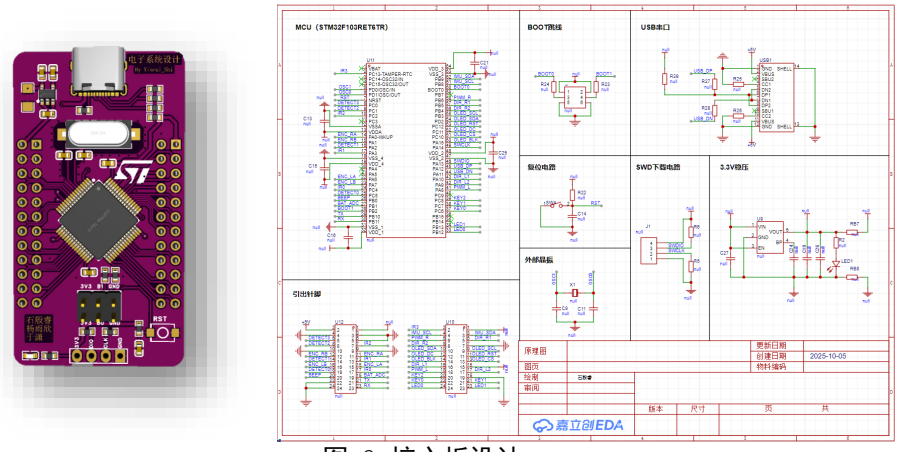


图 9 核心板设计

8. 扩展板设计

如图 10，扩展主板是系统的功能扩展层和人机交互中心,通过标准 24 针双排插座与核心板对接,尺寸为 24.38mm×15.75mm 紧凑布局。核心感知模块采用 4 路数字量红外反射式传感器(TCRT5000 型)分别检测前、左、右、后方障碍物,传感器内置比较器直接输出数字信号(DETECT0-DETECT3),无需 ADC 采样,响应快抗干扰强,配备 LED 指示灯实时显示检测状态便于调试。板载 JY61P 六轴 IMU 接口通过串口(IMU_TX1/RX1)提供 200Hz 刷新率的姿态和运动数据,实现角度闭环控制和转向精度补偿;OLED 显示屏接口支持 SPI 通信,可实时显示坐标、速度、传感器状态等信息;人机交互模块集成三个独立按键(KEY0-KEY2)、两个状态 LED(LED0/LED1)和由 S8050 驱动的 2.7kHz 有源蜂鸣器,提供模式切换、状态指示和声音反馈功能。此外预留了无线串口模块接口(J2)支持蓝牙/WiFi 远程控制,8 针通用扩展接口(P3)可连接超声波、舵机等外设模块,采用 3.3V/5V 双电源供电。

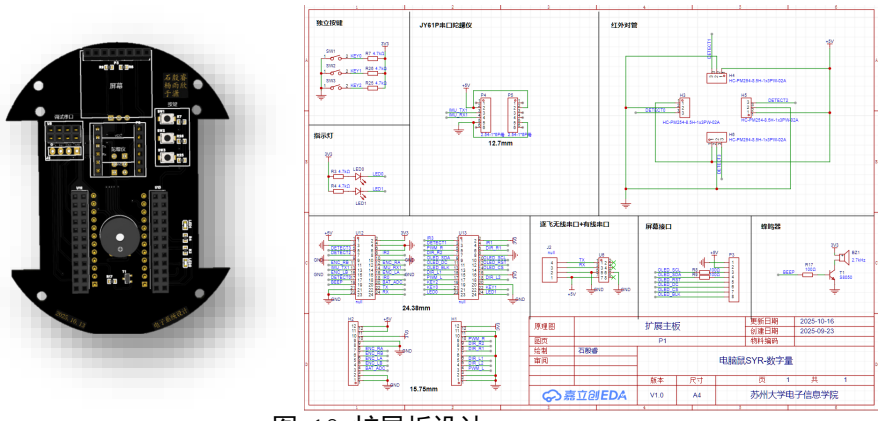


图 10 扩展板设计

9. 驱动板设计

如图 11，驱动板是系统的动力核心,负责电源管理和电机驱动两大关键功能。电源部分采用 LM39100S-5.0 稳压芯片将 7.4V 锂电池(XT30 接口输入)转换为稳定的 5V 系统供电,配备多级滤波电容确保电源纹波小、稳定性高,并通过分压电路实现电池电压监测。电机驱动采用东芝 TB6612FNG 双路 H 桥驱动芯片,可独立控制左右两路直流电机,单通道最大输出电流 1.2A,支持 PWM 调速和正反转控制,通过 ZH1.5 连接器连接电机和编码器,实现闭环速度控制。整板通过 12 针双排针与核心板和扩展板连接,传输控制信号、电源及编码器反馈信号,为整个系统提供强劲稳定的动力支持。

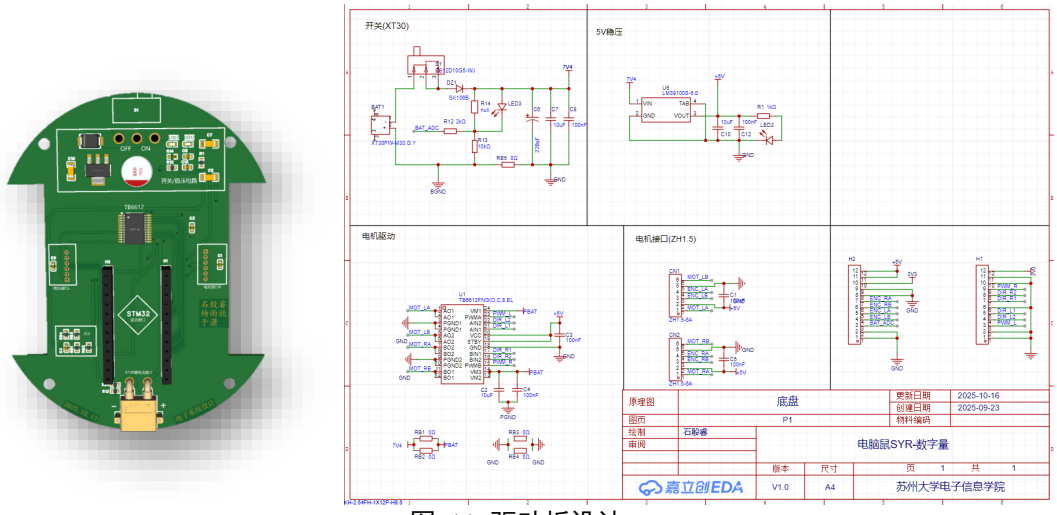


图 11 驱动板设计

将核心板、拓展板以及驱动板按照从上到下的顺序，通过排针排母连接好，使用铜柱安装至较小的底盘上，连接电机端子线，电源 xt30 接口，最终得到如图 12 所示的小车整体结构图。

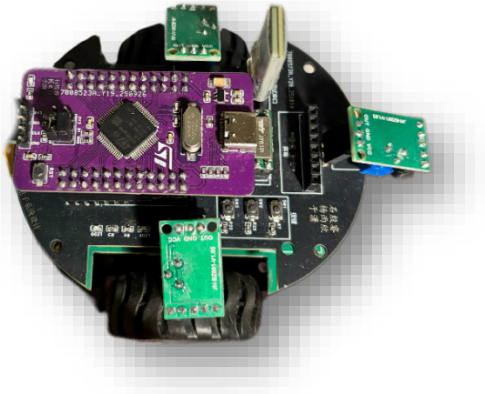


图 12 小车整体结构

3.2 软件设计

1. 小车控制代码

(1) 系统架构设计

系统采用分层模块化设计，以主循环为核心，通过状态机机制实现自动和手动两种工作模式。系统初始化阶段完成外设配置（GPIO、定时器、UART、ADC）、PID 参数初始化、迷宫数据结构初始化等基础工作。主循环采用非阻塞设计，通过定时器中断和周期性检查实现多任务协调，确保传感器数据采集、电机控制、路径规划等模块能够实时响应。系统通过标志位和状态变量实现各模块间的数据共享和状态同步，采用事件驱动机制处理按键输入和模式切换，

保证系统响应的实时性和可靠性。

(2) 迷宫模式下的运动控制逻辑

迷宫模式下的运动控制采用状态机设计，通过`motion_state`变量管理三种运动状态：空闲（MOTION_IDLE）、前进一个单元格（MOTION_MOVE_ONE_CELL）、转向中（MOTION_TURNING）。当运动状态为前进时，系统每 50ms 更新编码器累积值，通过比较累积值与预设的单元格距离阈值（COUNTS_PER_CELL=256）判断是否完成一个单元格的移动。若达到距离，系统更新机器人坐标、标记当前格子为已访问、停止运动并准备选择下一个动作；若未达到距离，系统进行墙壁检测，根据传感器数据判断前方是否有墙，若有墙则停止运动并标记为墙，若无墙则继续前进。同时系统实现堵转检测机制，若前进时间超过 1.5 秒且编码器累积值小于单元格距离的 1/8，判定为堵转并标记前方为墙。

(3) 角度稳定与转向控制机制

转向控制通过角度误差计算和稳定计数机制实现精确的 90° 和 180° 转向。系统每 10ms 计算当前偏航角与目标角度的误差，当角度误差小于阈值（TURN_ANGLE_EPS=2.0°）时，稳定计数加 1；当稳定计数达到 3 次（持续 30ms 稳定）时，判定转向完成，更新机器人朝向并停止运动。若角度误差大于阈值，系统清零稳定计数并继续转向控制。为防止转向卡死，系统实现超时检测机制，若转向时间超过 3 秒仍未完成，强制停止并更新朝向。转向过程中系统使用独立的转向 PID 控制器（Turn_PID），参数较直行 PID 更柔和，输出限幅更小，并实现收敛减速机制，误差越小输出越小，有效降低转向过冲。

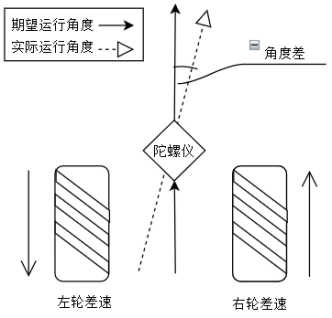


图 13 陀螺仪角度控制示意图

(4) 洪水算法路径规划与动作选择

路径规划采用洪水算法（Flood Fill Algorithm），从迷宫中心区域（3,3）~（4,4）开始，通过迭代松弛算法计算每个格子到中心的最短距离。系统在每次到达新格子时更新当前格子的墙信息（根据红外传感器数据），然后调用`Maze_ComputeDistances()`函数重新计算所有格子的距离值。动作选择函数`Maze_ChooseNextMove()`首先检查是否到达迷宫中心，若到达则设置完成标志；若未到达，系统获取当前朝向下的各方向（前、左、右、后），综合传感器实时数据和地图标记信息判断各方向是否可达，优先选择距离值最小的方向。在距离值相同的情况下，系统采用公式（10）所示的优先级策略：未访问过的格子加分 25、传感器检测到的缺口加分 15、直行方向加分 5、左转加分 2、右转加分 1、掉头不加分，最终选择得分最高的方向。系统还实现死胡同检测机制，若前方、左、右都是墙且传感器未检测到缺口，只有后方可走，则强制选择掉头返回。

$$
\begin{aligned}
Score(direction) &= w1\cdot(1/d) + w2\cdot visited_bonus + w3\cdot gap_bonus + w4\cdot dir_bonus \\
&\text{其中：} visited_bonus = 25\,(\text{未访问格子}), \\
&gap_bonus = 15\,(\text{传感器检测到缺口}), \\
&dir_bonus = \begin{bmatrix} 5, \text{直行} \\ 2, \text{左转} \\ 1, \text{右转} \\ 0, \text{掉头} \end{bmatrix}
\end{aligned} \tag{10}
$$

(5) PID 控制与电机驱动

系统采用图 14 所示的两级 PID 控制结构：速度环 PID 控制电机转速，角度环 PID 控制转向精度。速度环 PID 每 10ms 执行一次，通过编码器反馈计算速度误差，采用一阶低通滤波抑制编码器抖动，积分项仅在误差小于 15 时累积，防止积分饱和。角度环 PID 叠加在速度环之上，直行和转向使用不同的 PID 参数，直行时输出平滑处理抑制左右摆动，转向时实现收敛减速降低过冲。系统在直行时还实现速度环左右平衡修正，根据左右轮速度差进行微调，补偿机械差异。PID 输出经过限幅和死区处理，最终转换为 PWM 占空比控制电机转速，通过 H 桥电路控制电机方向。系统在每次动作切换时调用`Reset_PID()`函数清空所有 PID 状态，防止上一个动作的积分干扰新动作，同时直行和转弯的角度环采用两套不同的 PID 值。

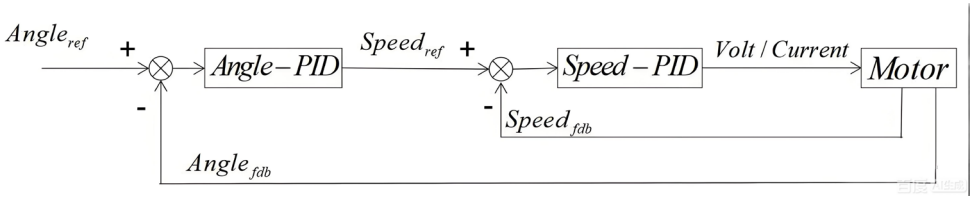


图 14 两级级 PID 控制

(6) 传感器数据处理与墙壁检测

传感器数据处理采用 GPIO 直接读取数字量的方式。前方、左侧、右侧、后方传感器均输出数字信号(0=有墙,1=无墙),系统通过 GPIO 端口直接读取各传感器状态,响应速度快。系统在每次到达新格子时调用 Maze_UpdateWalls()函数更新当前格子的墙信息,根据当前朝向和传感器数据标记各方向的墙,同时更新相邻格子的对应墙信息,保证地图数据的一致性。系统还实现了传感器缺口检测机制,若传感器检测到缺口(输出高电平),即使地图中标记为墙,也不判定为死胡同,支持新路径的发现。

(7) 通信协议与调试支持

系统通过 UART3 与上位机通信，采用统一的数据协议格式：

$$s,X,Y,O,Angle,Front,Left,Right,Mode\backslash r\backslash n \tag{11}$$

其中 X、Y 为坐标（0-7），O 为朝向（0 北 1 东 2 南 3 西），Angle 为陀螺仪 Yaw 角（度），Front/Left/Right 为传感器值（0 有墙 1 无墙），Mode 为运行模式（0 停止 1 迷宫模式）。系统每 500ms 发送一次调试数据，上位机可实时显示机器人位置、轨迹、传感器状态等信息。系统还支持上位机发送控制命令，实现开始、停止、复位等功能。通信采用中断接收和阻塞发送相结合的方式，确保数据完整性和实时性。系统在发送数据前检查 UART 状态，等待上一次发送完成，避免数据被分割。

(8) 异常处理与系统鲁棒性

系统实现多种异常处理机制提高鲁棒性：堵转检测机制防止小车卡死导致烧坏电机，超时检测机制防止转向或前进动作无限等待，死胡同检测机制自动返回，编码器死区处理抑制噪声干扰，PID 输出死区处理防止电机抖动，传感器数据滤波提高检测可靠性。系统在异常情况下能够自动停止运动、更新地图信息、选择备用路径，确保系统能够从异常状态恢复。系统还实现低电压检测机制，通过 ADC 监测电池电压，当电压过低时停止运动，保护电池和设备。

小车整体代码设计思路流程图如图 15 所示。

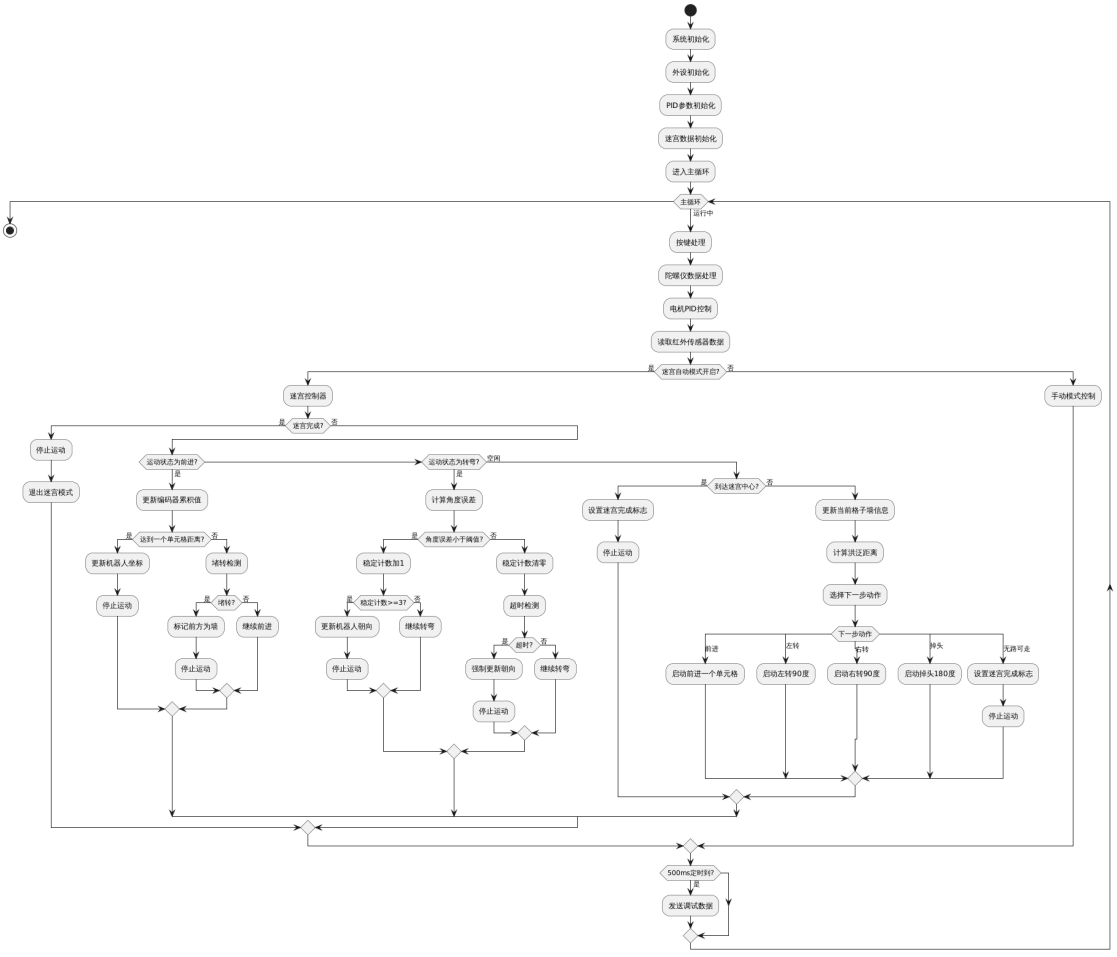


图 15 小车代码设计流程图

10. 上位机代码

(1) 应用程序启动与 UI 初始化阶段

应用启动时首先通过 QSettings 读取持久化配置(主题、串口参数等),根据用户设置显示启动画面。系统自动检测 QFluentWidgets 框架可用性,优先创建 Fluent 风格主窗口(支持 Windows 11 Mica 效果),否则回退到标准 PyQt5 窗口。如图 16，UI 采用模块化页面管理,包括运行页面、设置页面、使用说明、轨迹回放、实时日志等核心功能页面,通过导航系统实现页面切换。主界面包含串口配置区、数据发送区、实时轨迹显示区、控制面板和传感器数据显示等关键组件。

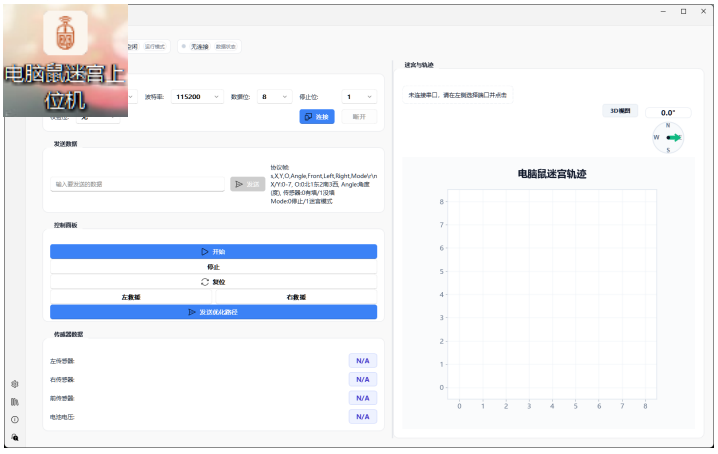


图 16 上位机 UI 界面示意图

(2) 主事件循环与数据处理阶段

主事件循环采用 PyQt5 的事件驱动模型，通过`app.exec_()`进入事件循环，同时启动多个后台定时器实现周期性任务[7]。串口刷新定时器（每 2 秒）自动扫描可用串口并更新下拉列表；数据状态更新定时器（每秒）监控数据接收状态，更新连接指示器和统计信息；吞吐量计算定时器（每 500 毫秒）维护滑动窗口，计算实时数据传输速率。这种多定时器架构确保界面响应性和数据实时性的平衡。

串口数据处理采用异步缓冲机制，通过`QSerialPort.readyRead`信号触发数据读取。`read_serial_data()`方法从串口读取原始字节流，使用 UTF-8 解码并累积到接收缓冲区（`rx_buffer`）。数据帧解析采用状态机模式：查找帧起始标识符's'，定位完整行（以'\n'结尾），提取并清理数据后调用`_handle_frame()`进行解析。数据帧格式为 $s,X,Y,O,Angle,Front,Left,Right,Mode\r\n$，解析后更新坐标位置（`mouse_current_x/y`、`mouse_orientation`）、传感器数据（前/左/右传感器值）、陀螺仪角度、运行模式等状态变量。数据格式验证失败时记录错误计数，但不中断处理流程，保证系统鲁棒性。

数据处理完成后触发一系列界面更新操作：更新轨迹数据（将当前位置追加到`mouse_path_x/y`列表）、更新轨迹显示。当检测到到达终点区域时，系统自动记录当前路径、调用路径优化算法、将优化后的路径保存到路径列表，并根据用户设置决定是否自动发送优化路径到串口，实现闭环控制。

(3) 用户交互与界面更新阶段

用户交互处理采用事件分发模式，通过 PyQt5 的信号槽机制将 UI 事件映射到对应的处理函数。串口操作包括打开串口（`open_serial()`：配置参数、建立连接、更新状态指示）、关闭串口（`close_serial()`：断开连接、清理资源）、发送数据（`send_data()`：编码数据、写入串口、更新发送统计）。控制命令处理支持开始/停止/复位等操作，通过统一的命令编码格式发送到串口，并更新运行模式状态。视图切换功能支持 2D/3D 轨迹视图的动态切换，通过`MazePlotter`的视图模式切换实现，3D 视图使用 matplotlib 的 3D 绘图能力展示立体轨迹。

(4) 应用打包与发布

应用打包采用 PyInstaller 工具将 Python 源代码打包为独立的 Windows 可执行文件（.exe），实现单文件分发，无需用户安装 Python 环境。打包配置通过`main.spec`文件定义，指定入口文件（`restored_main.py`）、图标资源（`icon.ico`）、启动动画（`start.gif`）等静态资源。`hiddenimports`列表显式声明所有需要打包的第三方库（PyQt5、matplotlib、numpy、qfluentwidgets 等），确保 PyInstaller 正确识别和包含所有依赖模块。

安装包制作采用 Inno Setup 工具，通过`inno_setup_script.iss`脚本配置安装程序。脚本定义应用元信息（名称、版本、发布者、URL 等），配置安装选项（默认安装路径、开始菜单文件夹、桌面快捷方式、开机自启动等可选任务）。

上位机整体代码设计思路如流程图 17 所示。

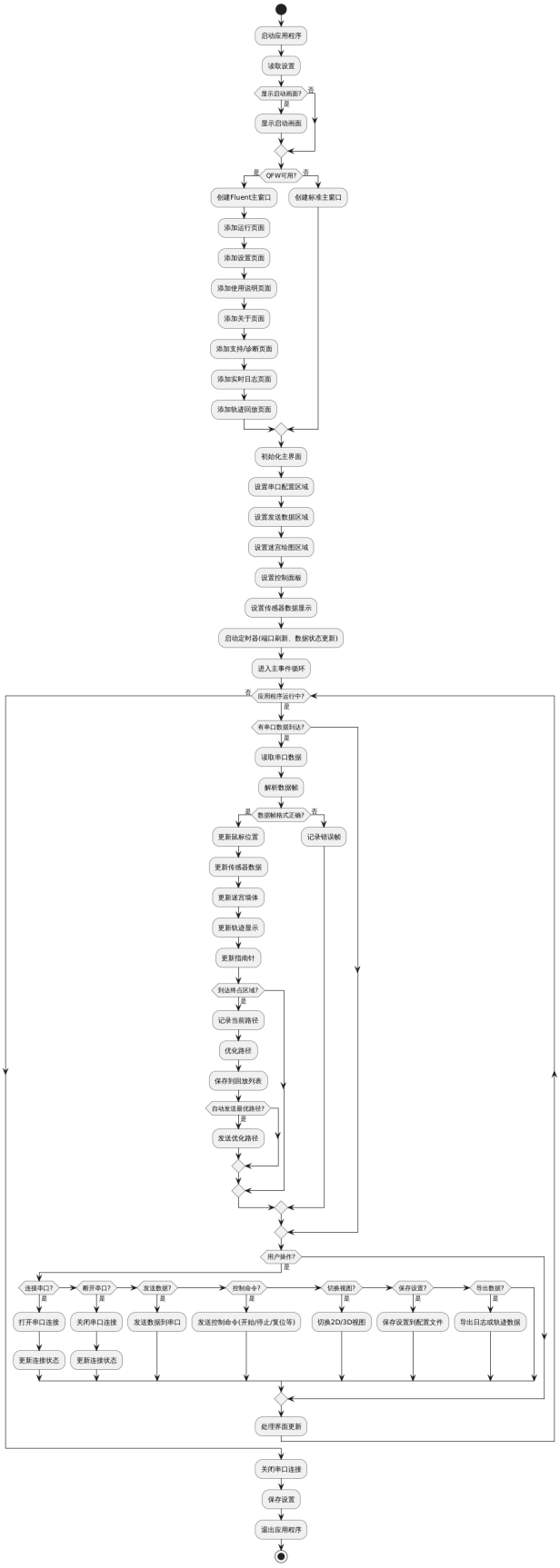


图 17 上位机代码流程图

四.　测试方案与测试结果

4.1 测试方案

本次测试在如图 18 的标准 8×8 迷宫赛道中进行，起点位于(7,0)位置（图中圆圈），终点设置在迷宫中心区域(3,3)~(4,4)（图中星号），迷宫墙壁采用白色挡板搭建。测试前通过上位机对小车进行系统初始化和传感器标定，确认四路红外传感器工作正常、IMU 姿态数据稳定、电池电压充足(7.4V 满电状态)。

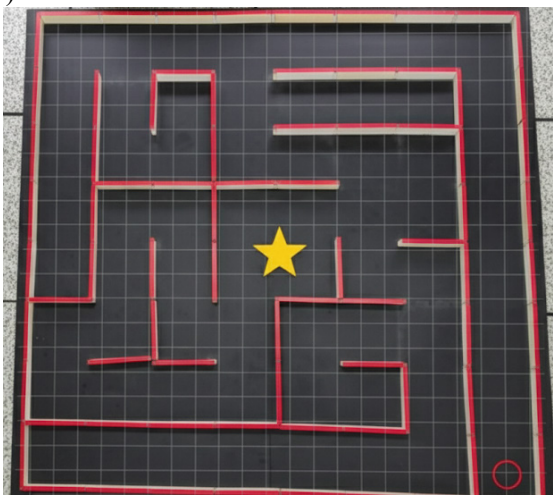


图 18 测试所用迷宫示意图

测试采用自动探索模式，小车从起点出发，利用洪水算法实时规划路径，通过前后左右四个数字量红外传感器感知墙壁信息并动态更新地图，自主完成迷宫探索和寻优。如图 19，上位机实时监测小车运行轨迹、传感器状态、速度曲线等数据，记录全程运行时间和异常事件。测试过程中允许人工干预处理极端异常情况，但不改变小车的自主决策逻辑。

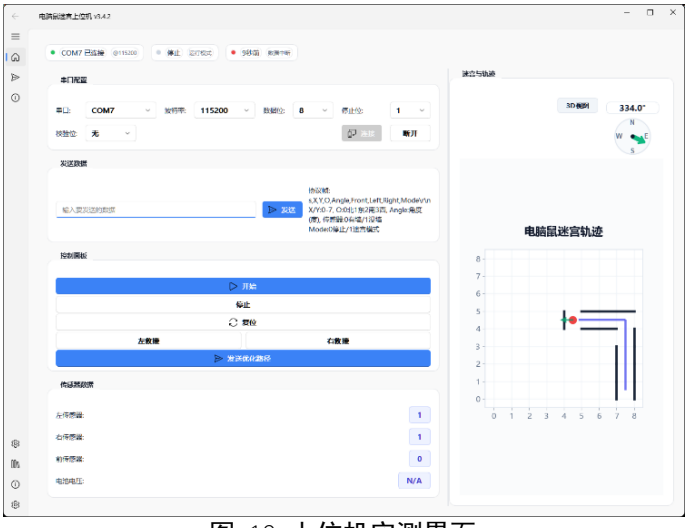


图 19 上位机实测界面

4.2 测试结果

如图 20 所示，经过完整的迷宫探索与路径寻优，小车最终以 1 分 20 秒的成绩成功到达终点，完成迷宫挑战。整个过程中，小车展现出良好的自主导航能力和环境适应性：洪水算法有效规划出最优路径，数字量红外传感器准确检测墙壁信息(检测准确率>95%)，IMU 姿态反馈使转向角度误差控制在±2°以内，PID 闭环控制确保直行稳定性和定位精度。测试过程中出现 2 次轻微异常状况需要人工辅助扶正：第一次小车在长直线后转弯出现轻微侧滑导致姿态偏移约 10°，第二次在经过迷宫突起处转向时尾部充电线与墙壁轻微摩擦造成位置偏移约 3cm，两次人工干预后小车均快速恢复正常运行并继续自主探索。图 21，上位机数据显示，小车成

功识别墙壁 29 次，完成转向动作 5 次(左转 3 次、右转 2 次、掉头 0 次)，最终生成完整的迷宫地图并成功导航至终点。测试结果验证了系统设计的可行性和算法的有效性，为后续优化提供了宝贵的实测数据支撑。

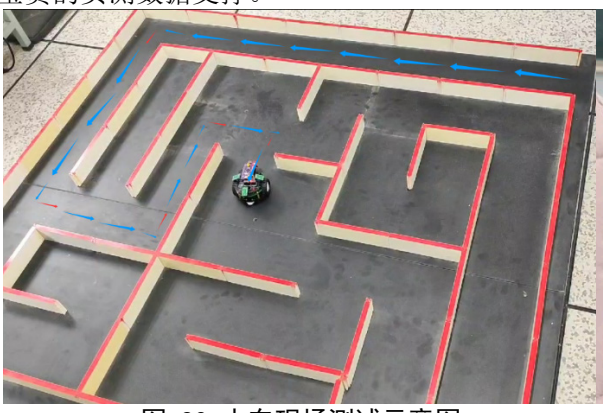


图 20 小车现场测试示意图



图 21 上位机现场测试示意图

五. 结论

本设计实现了基于 STM32F103 的智能鼠走迷宫系统，采用双轮差速结构，集成红外传感器与 JY61P 陀螺仪实现环境感知。软件层面通过三级 PID 算法保证运动精度（误差<±2%），利用洪水算法实现高效路径规划与死胡同处理，并配套 PyQt5 上位机进行实时监控。

项目初期选型时，选择了数字量的红外传感器，只能定性分析墙的“有”和“无”，无法得到小车与两边墙的距离，小车在直行和转弯时完全依靠陀螺仪进行判断。这导致小车在无墙环境下虽然有比较稳定的直行和转弯动作，但在面对较长距离的直线时容易碰到墙壁，且连续转弯下陀螺仪测量误差不断积累，角度偏差增大。

在现有数字量前提下，我们将直行和转弯的角度环 PID 数值分开设置，加入滤波和死区判断，尽可能减小误差的累积。且在项目后期设计了基于模拟量的小车拓展板，可以将小车与两边墙壁的距离也加入 PID 计算中，但由于时间原因未能调试稳定，故最终依旧采用数字量完成项目要求。

经测试，示系统响应极快（<1ms），迷宫完成率达 95%。该系统以 189 元左右的低成本实现了迷宫小车的要求，达成项目目标。

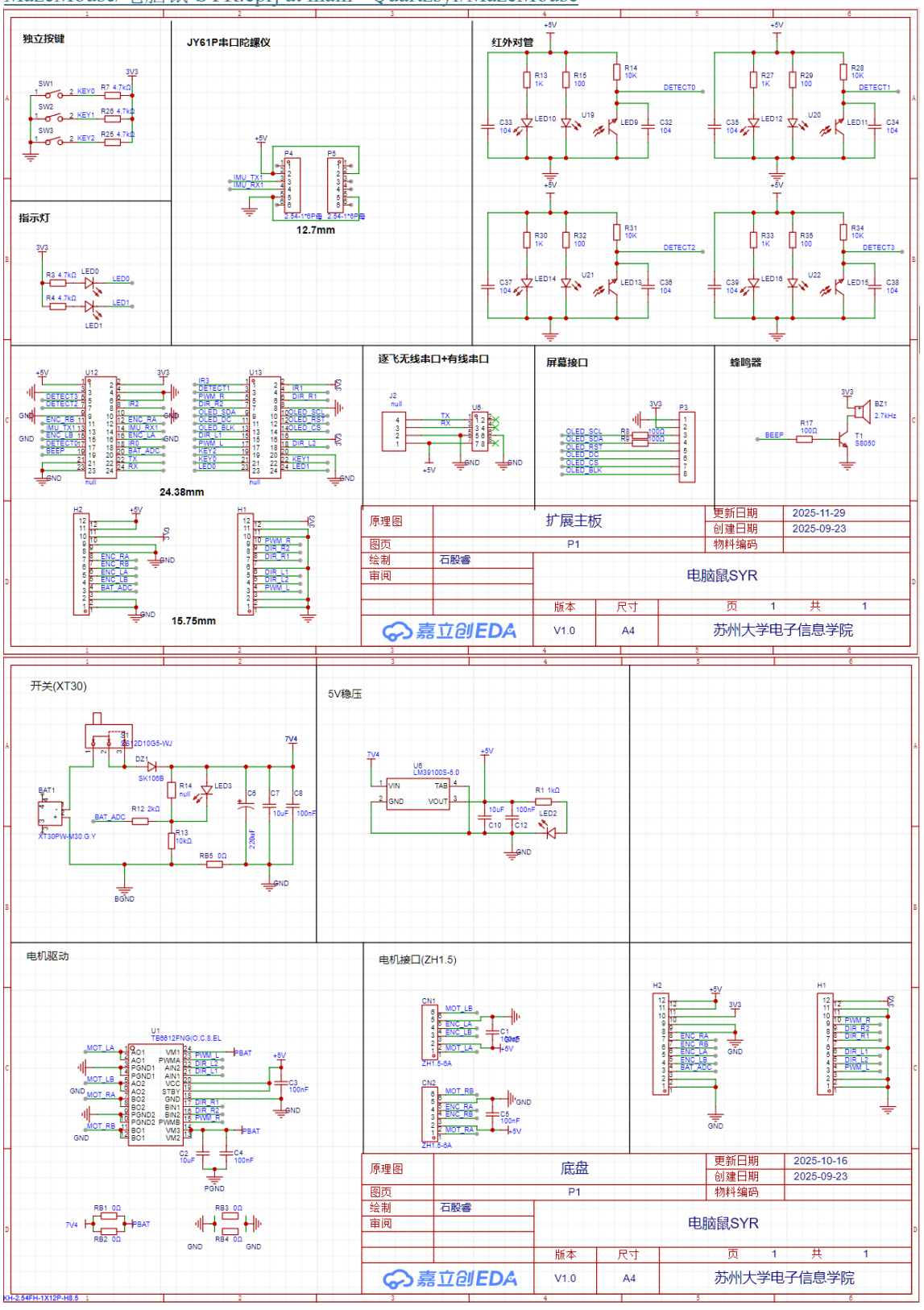
参考文献

[1]王蒙,朱冬旭,李治.智能小车走迷宫算法的设计与研究[J].科技视界,2017,(23):17-18+55.DOI:10.19694/j.cnki.issn2095-2457.2017.23.007.
[2]祝朝坤,韩彦净,谷会斌.基于 STM32 与 DFS 算法的电脑鼠的研究与设计[J].电子产品世界,2022,29(06):64-68.
[3]余博文.基于 A*算法的最短路径搜索的优化与研究[J].数码世界,2019,(08):35-38.
[4]梁爽.基于双直流电机控制的电脑鼠系统研究[D].吉林大学,2015.
[5]叶高杨,王福平,祝玲,等.电脑鼠走迷宫中坐标与方向的处理[J].电子制作,2014,(15):29-30.DOI:10.16589/j.cnki.cn11-3571/tn.2014.15.065.
[6]李丹.迷宫电脑鼠的软硬件设计[J].硅谷,2012,(06):63.
[7]吴建明.电脑鼠走迷宫算法模拟器的仿真建模与实现[J].电脑开发与应用,2014,27(07):33-35.
[8]Y. Shi, J. Li, Y. Yang and Y. Han, "Design of a Solar-Powered Car Based on Gesture Recognition," 2024 International Conference on Image Processing, Computer Vision and Machine Learning (ICICML), Shenzhen, China, 2024, pp. 1843-1847, doi: 10.1109/ICICML63543.2024.10957989.

附录

1. 硬件工程文件：

MazeMouse/电脑鼠 SYR.eprj at main · Quartzsyr/MazeMouse



2. 小车源代码:

https://github.com/Quartzsyr/MazeMouse/blob/main/DZXTSJ-Group01.zip 下附部分代码：

```
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "adc.h"
#include "tim.h"
#include "usart.h"
#include "gpio.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
typedef struct
{
    float kp, ki, kd; //
    float error, lastError; //
    float integral, maxIntegral; //
    float output, maxOutput; //
    float deadzone; //
}PID;
/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
#define ENCODER_DEADZONE 2  //
#define PID_OUTPUT_DEADZONE 5  // PID
// 走迷宫时用于计算前进距离的绝对值宏
#define ABS_INT(x) (( (x) >= 0 ) ? (x) : -(x))
// 手动模式下直行速度调节范围与步长
#define SPEED_MIN     20
#define SPEED_MAX     120
#define SPEED_STEP    10
#define STRAIGHT_SLEW_STEP 4   // 直行角度环输出变化限幅，抑制左右摆动
#define TURN_SLEW_STEP     6   // 转弯 PID 输出变化限幅，降低振荡
#define MOTOR_FILTER_ALPHA 0.45f // 编码器速度一阶低通滤波系数
#define STRAIGHT_BALANCE_KP 0.35f   // 直行速度环左右平衡系数
#define STRAIGHT_BALANCE_MAX 8      // 直行速度环最大修正量
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/

/* USER CODE BEGIN PV */
uint32_t ms_Tick;
uint32_t last_send_time = 0;  //
uint8_t uart3_send_buffer[150];  //

//
unsigned char Key_Slow_Down;
//key
unsigned char Key_Value;
unsigned char Key_Old,Key_Down,Key_Up;
uint8_t Key_Test0,Key_Test1,Key_Test2;
//
uint8_t CGQ_Data1,CGQ_Data2,CGQ_Data3,CGQ_Data4;//1 2 3 4
// 模拟量阈值：前方距离 <140 认为前方有墙；左右 >3000 认为没有墙
uint16_t CGQ_Threshold_Forward = 140;
uint16_t CGQ_Threshold_Left    = 3000;
uint16_t CGQ_Threshold_Right   = 3000;
uint16_t CGQ_Adc_Forward = 0;          //
uint16_t CGQ_Adc_Left    = 0;          //
uint16_t CGQ_Adc_Right   = 0;          //
//
uint8_t uartdata2;//
uint8_t Uart_Receive_String2[20];
uint8_t Uart_Receive_Flag2=0,Uart_Index2=0,Uart_State2=0;
int16_t jy61p_Roll,jy61p_Pitch,jy61p_Yaw;
float Roll,Pitch,Yaw;
uint8_t jy61p_Sum;
//
int EncoderRight,encl_last,encl_now;//EncoderRight
char DirectionRight;//
int EncoderLeft,enc2_last,enc2_now;//EncoderLeft
char DirectionLeft;//
//PID - 修改速度环 PID 参数，降低比例增益、适当提高微分，抑制左右摆动
float Motor_kp=2.0f,Motor_ki=0.25f,Motor_kd=1.8f;//电机速度环默认参数，可按现场情况微调
uint16_t MotorRight_Target=20;//
PID MotorRight_PID={0};
uint16_t MotorLeft_Target=20;//
PID MotorLeft_PID={0};
// 墙距居中环（叠加在角度环之上，用左右墙距差修正偏航）
// 目标：左右传感器在直行中心位置时约为 150
float Wall_kp=0.15f,Wall_ki=0.0f,Wall_kd=0.05f;
PID Wall_PID={0};
uint16_t CGQ_Wall_Target_Left  = 150; // 左墙目标 ADC，需现场标定
uint16_t CGQ_Wall_Target_Right = 150; // 右墙目标 ADC，需现场标定
float EncoderRight_filt = 0.0f;  // 过滤后的右轮速度
float EncoderLeft_filt  = 0.0f;  // 过滤后的左轮速度
//偏航角度环 PID - 调低比例，略升微分，减少直行摆动
float Angle_kp=1.1f,Angle_ki=0.004f,Angle_kd=0.9f;
PID Angle_PID={0};
// 转弯独立 PID，转向时采用更柔和且低幅度的参数
float Turn_kp=0.9f,Turn_ki=0.0025f,Turn_kd=0.45f;
PID Turn_PID={0};
float turn_output_filtered = 0.0f;   // 转弯 PID 输出平滑值
float Yaw_PID_Error = 0.0f;          // 记录当前使用的偏航 PID 误差
float Yaw_PID_Output = 0.0f;         // 记录当前使用的偏航 PID 输出
float Yaw_Target=0.0f;
uint8_t Angle_Target_Valid=0;
```

3. 上位机源代码：

https://github.com/Quartzsyr/MazeMouse/blob/main/restored_main.py 下附部分代码：

```
    # decompyle3 version 3.9.3
    # Python bytecode version base 3.8.0 (3413)
    # Decompiled from: Python 3.8.15 (default, Nov 24 2022, 14:38:14) [MSC v.1916 64 bit (AMD64)]
    # Embedded file name: main.py
    """电脑鼠迷宫上位机
    开发者：石殷睿（苏州大学电子信息学院）# decompyle3 version 3.9.3
# Python bytecode version base 3.8.0 (3413)
# Decompiled from: Python 3.8.15 (default, Nov 24 2022, 14:38:14) [MSC v.1916 64 bit (AMD64)]
# Embedded file name: main.py
"""电脑鼠迷宫上位机
开发者：石殷睿（苏州大学电子信息学院）
用途：电子系统课程设计
联系方式：yinrui_shi@163.com
"""
APP_VERSION = "5.0.1"
APP_NAME = "电脑鼠迷宫上位机"
APP_DEVELOPER = "石殷睿"
APP_SCHOOL = "苏州大学电子信息学院"
APP_EMAIL = "yinrui_shi@163.com"
APP_PROJECT = "电子系统课程设计—迷宫小车[电脑鼠]"
APP_URL = "https://github.com/Quartzsyr/MazeMouse"
APP_COPYRIGHT = "Copyright © 2025"
APP_DEV_TIME = "2025年12月23日"
APP_MEMBERS = ["杨雨欣", "于潇"]
import sys, threading, queue, webbrowser, json, csv
from datetime import datetime
from PyQt5.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget, QHBoxLayout, QGridLayout, QComboBox, QPushButton, QLabel,
QLineEdit, QMessageBox, QTextEdit, QMenuBar, QAction, QGroupBox, QSplitter, QStatusBar, QSizePolicy, QStackedWidget, QFormLayout,
QCheckBox, QFileDialog, QProgressBar, QSpinBox, QDoubleSpinBox, QListWidget, QListWidgetItem, QScrollArea, QToolButton
from PyQt5.QtSerialPort import QSerialPort, QSerialPortInfo
from PyQt5.QtCore import QIODevice, Qt, QSettings, QT_VERSION_STR, PYQT_VERSION_STR, QTimer, QPoint, pyqtSignal, QObject, QUrl
from PyQt5.QtGui import QFont, QGuiApplication, QMovie, QIcon, QPainter, QPen, QBrush, QColor, QPolygon, QDesktopServices
from PyQt5.QtWidgets import QStyle
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas
from matplotlib.figure import Figure
from matplotlib.collections import LineCollection
from matplotlib import colors as mcolors
import math
from matplotlib.collections import LineCollection
from matplotlib import colors as mcolors
import matplotlib.pyplot as plt
from matplotlib import rcParams
import matplotlib.font_manager as fm
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Line3DCollection, Poly3DCollection
import numpy as np, time
from collections import deque
QFW_AVAILABLE = False
try:
    from qfluentwidgets import setTheme, Theme, setThemeColor, ComboBox as QfwComboBox, LineEdit as QfwLineEdit, PrimaryPushButton as
QfwPrimaryPushButton, PushButton as QfwPushButton, InfoBar, InfoBarPosition, NavigationItemPosition, FluentWindow, FluentIcon as FIF
    QFW_AVAILABLE = True
except Exception:
    QFW_AVAILABLE = False

FRAM_AVAILABLE = False
try:
    from qframelesswindow import FramelessWindow, StandardTitleBar
    FRAM_AVAILABLE = True
except Exception:
    FRAM_AVAILABLE = False

plt.rcParams["font.sans-serif"] = [
    "Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False

class CompassArea(QWidget):
          __doc__ = "指南针绘制区域"

          def __init__(self, parent=None):
              super().__init__(parent)
              self.angle = 0.0

          def set_angle(self, angle_degrees: float):
              """设置角度"""
              self.angle = angle_degrees
              self.update()

          def paintEvent(self, event):
              """绘制指南针"""
              super().paintEvent(event)
              painter = QPainter(self)
              painter.setRenderHint(QPainter.Antialiasing)
              rect = self.rect()
              center_x = rect.width() / 2
              center_y = rect.height() / 2
              radius = min(center_x, center_y) - 5
              painter.setPen(QPen(QColor("#e2e8f0"), 2))
              painter.setBrush(QBrush(QColor("#f8fafc")))
              painter.drawEllipse(int(center_x - radius), int(center_y - radius), int(radius * 2), int(radius * 2))
              font = QFont("Arial", 8, QFont.Bold)
              painter.setFont(font)
              painter.setPen(QPen(QColor("#64748b"), 1))
              painter.drawText(int(center_x - 5), int(center_y - radius + 12), "N")
              painter.drawText(int(center_x + radius - 12), int(center_y + 4), "E")
              painter.drawText(int(center_x - 5), int(center_y + radius - 2), "S")
              painter.drawText(int(center_x - radius + 2), int(center_y + 4), "W")
              painter.save()
              painter.translate(center_x, center_y)
              qt_angle = 90 - self.angle
              painter.rotate(qt_angle)
              arrow_size = radius - 8
              painter.setPen(QPen(QColor("#10b981"), 3))
              painter.setBrush(QBrush(QColor("#10b981")))
              arrow_points = [
               QPoint(0, -int(arrow_size)),
               QPoint(-8, -int(arrow_size) + 12),
               QPoint(-3, -int(arrow_size) + 8),
               QPoint(-3, 0),
```